Autonomous Parking

1. Course Content

Note: This program runs on a sandbox map; you will need to adjust the program for your personal map.

1. Learn the actions the car takes after recognizing traffic lights.
2. Learn the newly introduced key source code.

2. Self-service parking source code

Self-service parking source code path:

```
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/auto_drive/yolo_detect.py
```

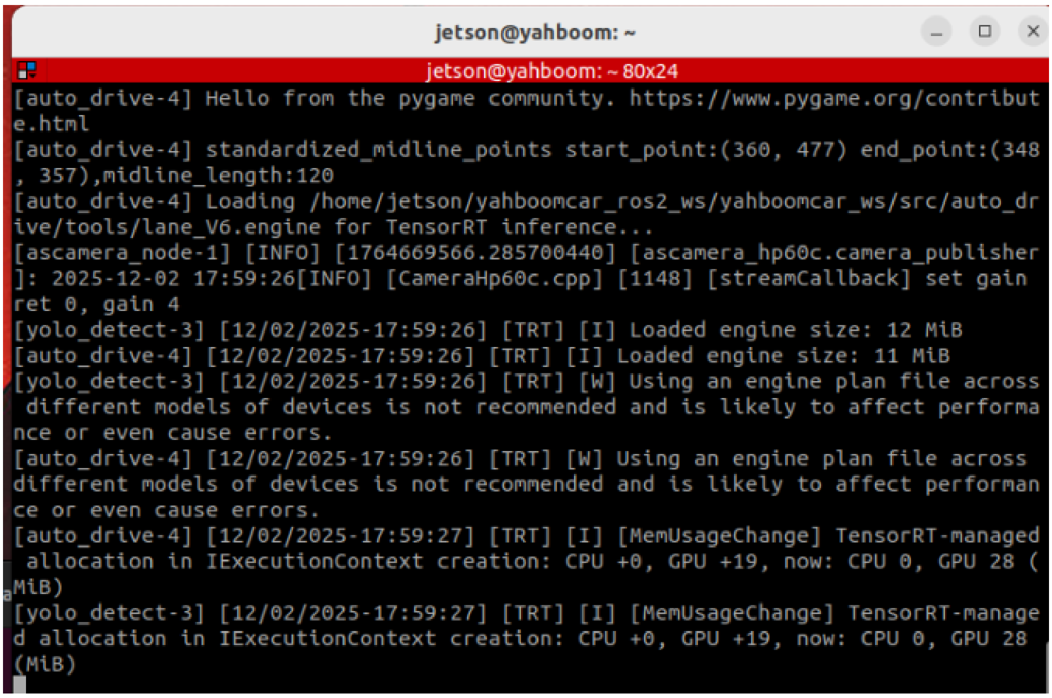
```
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/auto_drive/auto_drive.py
```

3. Program Startup

3.1 Startup Command

- Start camera + chassis + road signs and lane markings

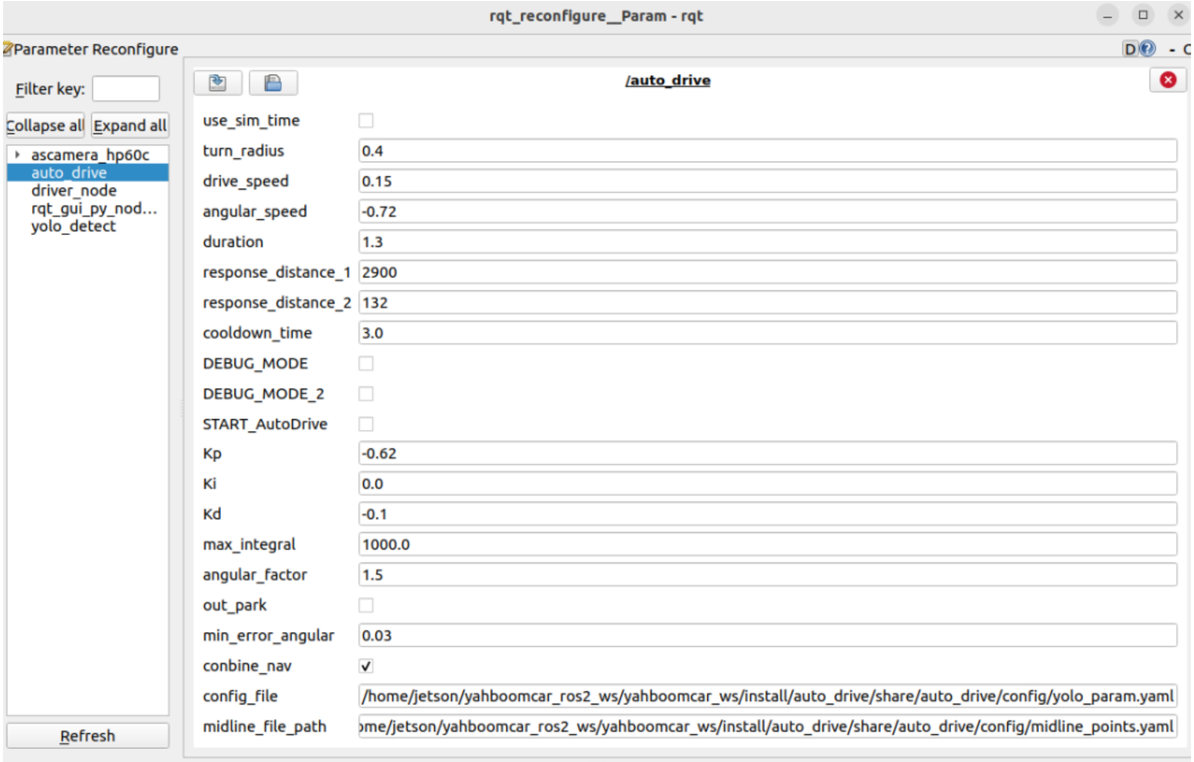
```
ros2 launch auto_drive auto_drive_bringup.launch.py
```



After the program runs, a YOLO recognition screen will be displayed. The car will move centered within the lane and perform corresponding actions based on the road signs it detects during its journey. (If the terminal does not report any errors after starting the program, but the car does not move, you can run the following program to check the parameters.)

- Start the dynamic parameter configuration node

```
ros2 run rqt_reconfigure rqt_reconfigure
```

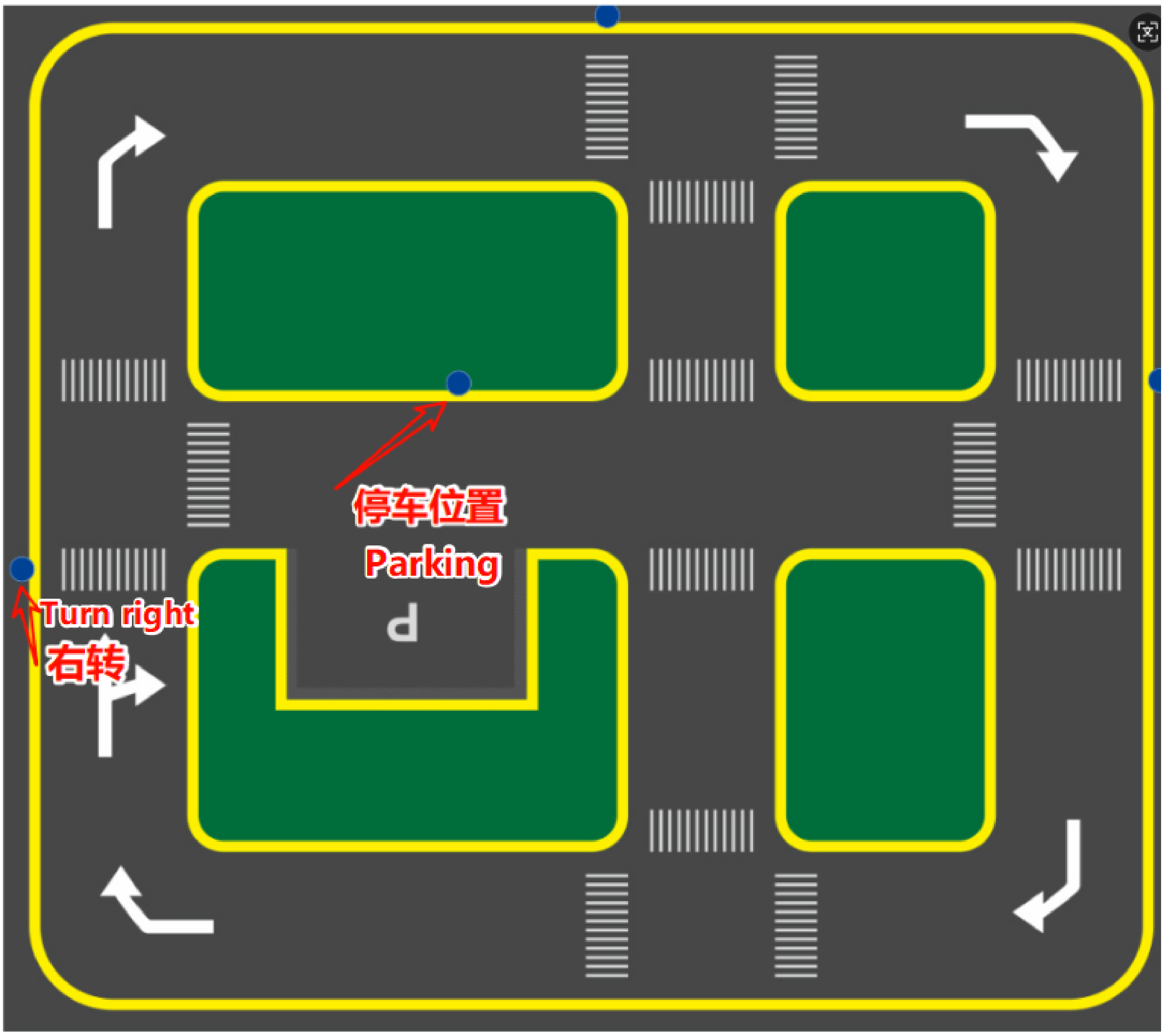


If the car doesn't move, check two parameters: whether START_AutoDrive is checked and whether conbine_nav is false.

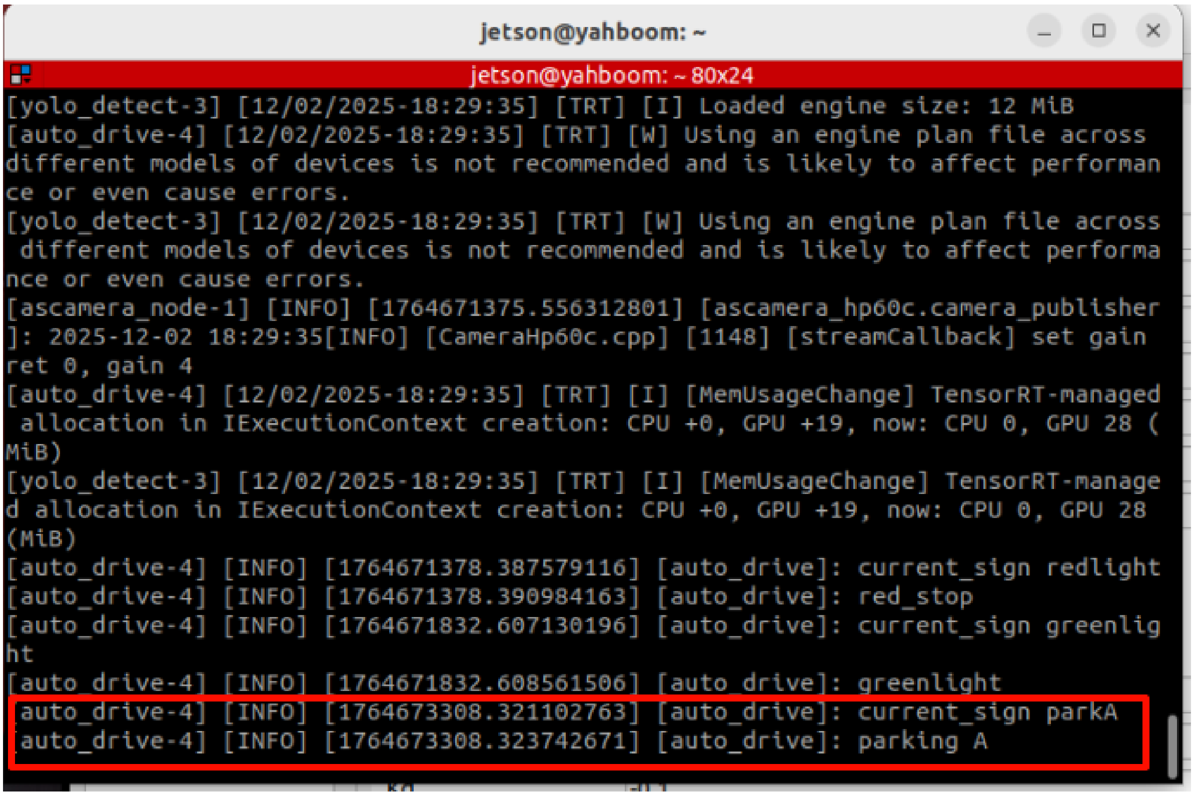
✅ After modifying the parameters, click in the blank area of the GUI to enter the parameter values. Note that this only applies to the current startup; for permanent effects, the parameters need to be modified in the source code.

3.2 Recognizing the Car's Movement After Parking

Note: Because self-parking uses fixed actions, the right turn and parking A signs need to be placed in fixed positions.



The program will print when it recognizes parking A (parking A and parking B perform the same action).



```
jetson@yahboom: ~
jetson@yahboom: ~ 80x24
[yolo_detect-3] [12/02/2025-18:29:35] [TRT] [I] Loaded engine size: 12 MiB
[auto_drive-4] [12/02/2025-18:29:35] [TRT] [W] Using an engine plan file across
different models of devices is not recommended and is likely to affect performan
ce or even cause errors.
[yolo_detect-3] [12/02/2025-18:29:35] [TRT] [W] Using an engine plan file across
 different models of devices is not recommended and is likely to affect performa
nce or even cause errors.
[ascamera_node-1] [INFO] [1764671375.556312801] [ascamera_hp60c.camera_publisher
]: 2025-12-02 18:29:35[INFO] [CameraHp60c.cpp] [1148] [streamCallback] set gain
ret 0, gain 4
[auto_drive-4] [12/02/2025-18:29:35] [TRT] [I] [MemUsageChange] TensorRT-managed
 allocation in IExecutionContext creation: CPU +0, GPU +19, now: CPU 0, GPU 28 (
MiB)
[yolo_detect-3] [12/02/2025-18:29:35] [TRT] [I] [MemUsageChange] TensorRT-manage
d allocation in IExecutionContext creation: CPU +0, GPU +19, now: CPU 0, GPU 28
(MiB)
[auto_drive-4] [INFO] [1764671378.387579116] [auto_drive]: current_sign redlight
[auto_drive-4] [INFO] [1764671378.390984163] [auto_drive]: red_stop
[auto_drive-4] [INFO] [1764671832.607130196] [auto_drive]: current_sign greenlig
ht
[auto_drive-4] [INFO] [1764671832.608561506] [auto_drive]: greenlight
[auto_drive-4] [INFO] [1764673308.321102763] [auto_drive]: current_sign parkA
[auto_drive-4] [INFO] [1764673308.323742671] [auto_drive]: parking A
```

The car will perform a fixed action to enter the parking space, moving forward for 3 seconds and then laterally into the space.

The self-service parking processing function is located in the `auto_drive.py` file. Below is the function that enters the parking space after the car is detected. Users can modify it themselves to allow the car to move after recognizing road signs.

```
def parking_A(self):
    ''''停车场A、B'''
    self.get_logger().info('parking A')
    time.sleep(5.0)
    self.turn_action_active = True
    self.__set_current_speed(linear_x=-self.drive_speed, angular_z=0.8)
    time.sleep(3.6)
    self.__set_current_speed(linear_x=-self.drive_speed, angular_z=-0.8)
    time.sleep(2.7)
    self.__set_current_speed(linear_x=self.drive_speed, angular_z=0.0)
    time.sleep(2.2)
    self.__set_current_speed(linear_x=0.0, angular_z=0.0)
```

4. Source Code Analysis

4.1 Auto_drive.py

YOLO signpost message reception:

```
def sign_callback(self, msg:DetectionObject):
    '''Subscribe to YOLO recognition results'''
    class_name = msg.class_name
    object_distance = self._get_distance(msg.ymin)  # Calculate distance
    object = Object(msg.class_name, msg.confidence, object_distance)
```

Multiple filtering mechanisms: mainly to prevent continuously recognizing road signs and triggering actions; the action will only be unlocked when different road signs are recognized.

```
# 1. Check if any other actions are being performed.
if self.action_runing:
    return

# 2. Check cooldown time (to prevent repeated triggering)
current_time = time.time()
if class_name in self.last_processed_time:
    time_diff = current_time - self.last_processed_time[class_name]
    if time_diff < self.cooldown_time:  # Default cooldown: 3 seconds
        return

# 3. Check if it is a duplicate road sign.
if class_name == self.last_sign:
    return

# 4. Check if the road sign is enabled.
if not self.sign_enable_list[对应的索引]:
    return
```

Event queue processing

```
def handle_events(self):
    '''Handling target recognition events'''
    while True:
        if not self.event_queue.empty():
            object = self.event_queue.get()
            self.action_runing = True
            current_sign = object.class_name

            try:
                # Perform the appropriate action based on the type of road sign.
                if object.class_name == "straight" and
self.sign_enable_list[10]:
                    self.go_straight()
                elif object.class_name == "turnright" and
self.sign_enable_list[11]:
                    self.turn('right')
                # ... Other road sign processing

            finally:
                # Update status record
                self.last_processed_time[current_sign] = time.time()
                self.last_sign = current_sign
                self.action_runing = False
```

Specific action implementation

```
def parking_A(self):
    ''''停车场A、B'''
    self.get_logger().info('parking A')
    time.sleep(5.0)
    self.turn_action_active = True
    self.__set_current_speed(linear_x=-self.drive_speed, angular_z=0.8)
    time.sleep(3.6)
    self.__set_current_speed(linear_x=-self.drive_speed, angular_z=-0.8)
    time.sleep(2.7)
    self.__set_current_speed(linear_x=self.drive_speed, angular_z=0.0)
    time.sleep(2.2)
    self.__set_current_speed(linear_x=0.0, angular_z=0.0)
```